# **Laboratorio 7:** Mini App de Perfil de Usuario

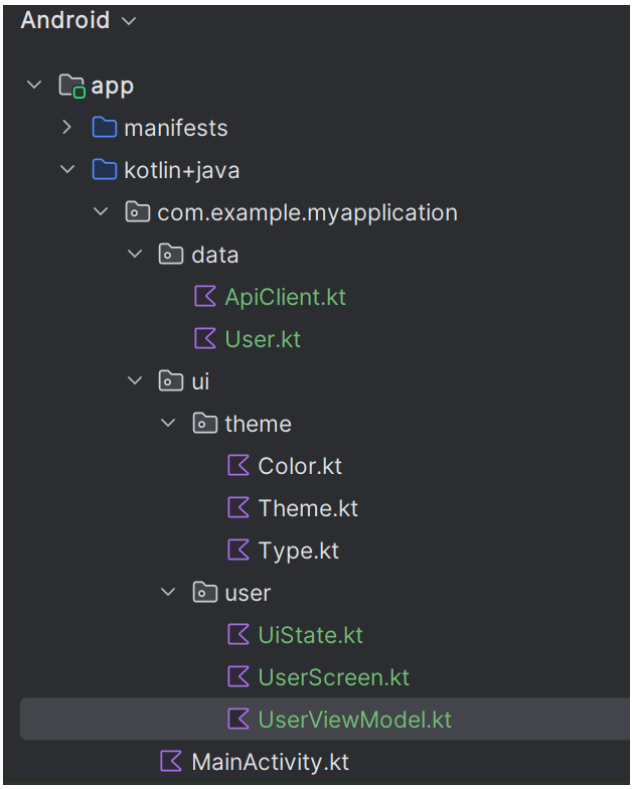
---

## 1. OBJETIVOS DE APRENDIZAJE

Construir una mini app que:

- Haga una “petición” simulada usando un object ApiClient.
- Reciba un modelo inmutable User (data class).
- Lo envuelva en una máquina de estados UiState (sealed class).
- Lo exponga desde un ViewModel usando StateFlow.
- Lo muestre en Compose usando un when exhaustivo.
- Use un companion object para validar la creación del usuario.

## 2. ESTRUCTURA FINAL DEL PROYECTO



Si la estructura de su proyecto no tiene un paquete que consta en la figura anterior, por ejemplo, 🗀 **user**, debe crearlo siguiendo los pasos:

a. En la vista **Project**, hacer clic derecho sobre:
   🗀 com.example.myapplication
b. Seleccionar:
   New ⟶ Package
c. Escribir:
   com.example.myapplication.ui.user
d. Android Studio automáticamente creará:
   ui/
     theme/
     user/

**¿Dónde colocar cada archivo?**

**Dentro de data/:**

- ApiClient.kt
- User.kt

**Dentro de ui/user/:**

- UiState.kt
- UserViewModel.kt
- UserScreen.kt

**MainActivity.kt** Se queda donde está.

**Nota**: Android Studio colapsa los paquetes que comparten prefijo. ui.theme no es una sola carpeta: es ui/theme. Por eso no se ve ui directamente, pero sí existe. Para crear subpaquetes, solo hay que escribir el nombre completo del paquete.

## 3. REQUISITOS PREVIOS

- Haber completado la estandarización del entorno (Laboratorio 1).
- Contar con el repositorio de GitHub vinculado (Laboratorio 2).

## 4. PASO A PASO DETALLADO

### Paso 1: Preparación del Proyecto (Herencia de Plantilla)

1. Localizar la carpeta del **Laboratorio 2** en el explorador de archivos del sistema operativo.

2. Copiar la carpeta y renombrarla como RDA1_Laboratorio7_NombreApellido.
3. **Limpieza**: Borrar las carpetas ocultas .gradle y build dentro de la nueva carpeta para evitar conflictos de caché.
4. Abrir Android Studio y seleccionar **File > Open**, buscando la nueva carpeta.
5. Navegar a settings.gradle.kts en la raíz del proyecto.
6. Modificar la línea: rootProject.name = "KotlinEcosystemLab".
7. Package: “com.example.lab”
8. Hacer clic en el icono del elefante (**Sync Project with Gradle Files**) y esperar a que finalice la sincronización.

## Paso 2: Crear el Singleton: ApiClient

🗀 **Ruta:** app/java/com.example.lab/data/ApiClient.kt

```
package com.example.lab.data  //Validar que este nombre coincida con su paquete

import kotlinx.coroutines.delay

object ApiClient {

    // Simula una llamada de red
    suspend fun fetchUser(): User {
        delay(1500) // simula latencia
        return User.fromEmail("estudiante@puce.edu.ec")
    }
}
```

🧠 **Pregunta de reflexión:**
¿Por qué ApiClient debe ser un *object* y no una clase normal?

## Paso 3: Crear el modelo inmutable: User (data class + companion)

🗀 Ruta: app/java/com.example.lab/data/User.kt

```
package com.example.lab.data  //Validar que este nombre coincida con su paquete

data class User private constructor(
    val id: String,
    val email: String
) {
    companion object {
        fun fromEmail(email: String): User {
            require(email.contains("@")) { "Email inválido" }
            val normalized = email.trim().lowercase()
            return User(
```

```
            id = "user_${System.currentTimeMillis()}",
            email = normalized
        )
    }
  }
}
```

🧠 Pregunta de reflexión:
¿Qué pasaría si el constructor no fuera privado?

(**Nota**: Revisar y validar los nombres de los paquetes al inicio de cada archivo. Pueda ser que el nombre de su paquete sea distinto al nombre en el código proporcionado en esta guía. Si su paquete tiene otro nombre, e.g., com.example.myapplication.data, colocar ese nombre en lugar del que está en el código proporcionado en esta guía, se recomienda dejar el nombre que se genera automáticamente el momento de crear el archivo .kt)

## Paso 4: Crear la máquina de estados: UiState

🗀 **Ruta:** app/java/com.example.lab/ui/UiState.kt

```
package com.example.lab.ui   //Validar que este nombre coincida con su paquete

import com.example.lab.data.User  //Validar que este nombre coincida con su paquete

sealed class UiState {
  data object Loading : UiState()
  data class Success(val user: User) : UiState()
  data class Error(val message: String) : UiState()
}
```

🧠 Pregunta de reflexión:
¿Por qué sealed class garantiza que el when sea exhaustivo?

## Paso 5: Crear el ViewModel con StateFlow

🗀 **Ruta:** app/java/com.example.lab/ui/UserViewModel.kt

```
package com.example.lab.ui   //Validar que este nombre coincida con su paquete

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.lab.data.ApiClient   //Validar que este nombre coincida con su paquete
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch

class UserViewModel : ViewModel() {
```

```
    private val _state = MutableStateFlow<UiState>(UiState.Loading)
    val state = _state.asStateFlow()

    init {
        loadUser()
    }

    private fun loadUser() {
        viewModelScope.launch {
            try {
                val user = ApiClient.fetchUser()
                _state.value = UiState.Success(user)
            } catch (e: Exception) {
                _state.value = UiState.Error("No se pudo cargar el usuario")
            }
        }
    }
}
```

🧠 Pregunta de reflexión:
¿Por qué usamos StateFlow en lugar de exponer directamente el User?

## Paso 6: Crear la UI en Compose

🗀 Ruta: app/java/com.example.lab/ui/UserScreen.kt

```
package com.example.lab.ui    //Validar que este nombre coincida con su paquete

import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import com.example.lab.data.User

@Composable
fun UserScreen(state: UiState) {
    when (state) {
        is UiState.Loading -> LoadingView()
        is UiState.Success -> UserView(state.user)
        is UiState.Error -> ErrorView(state.message)
    }
}
```

```
@Composable
fun LoadingView() {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        CircularProgressIndicator()
    }
}

@Composable
fun UserView(user: User) {
    Column(
        modifier = Modifier.fillMaxSize().padding(24.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Text("Usuario cargado:")
        Spacer(Modifier.height(8.dp))
        Text("ID: ${user.id}")
        Text("Email: ${user.email}")
    }
}

@Composable
fun ErrorView(message: String) {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        Text("Error: $message")
    }
}
```

🤔 **Pregunta de reflexión:**

¿Qué ventaja tiene usar un when exhaustivo en Compose?

## Paso 7: Conectar todo en MainActivity

🗀 **Ruta: app/java/com.example.lab/MainActivity.kt**

```
package com.example.lab    //Validar que este nombre coincida con su paquete

import androidx.compose.runtime.collectAsState
import android.os.Bundle
import androidx.activity.ComponentActivity
```

```
import androidx.activity.compose.setContent
import androidx.activity.viewModels
import com.example.lab.ui.UserScreen   //Validar que este nombre coincida con su paquete
import com.example.lab.ui.UserViewModel   //Validar que este nombre coincida con su paquete

class MainActivity : ComponentActivity() {

   private val vm: UserViewModel by viewModels()

   override fun onCreate(savedInstanceState: Bundle?) {
      super.onCreate(savedInstanceState)

      setContent {
         val state = vm.state.collectAsState().value
         UserScreen(state)
      }
   }
}
```

## 6. RESULTADOS ESPERADOS

Cuando se ejecute la aplicación:

a. Se deberá ver un loader.
b. Luego aparecerá el usuario cargado.
c. Todo usando el ecosistema Kotlin completo.

🧠 **Pregunta de reflexión:**

Con sus propias palabras, explique cómo fluye la información dentro de la aplicación que construyó en el laboratorio. Describir qué papel cumple cada uno de los archivos creados (ApiClient, User, UiState, UserViewModel, UserScreen, MainActivity) y cómo se relacionan entre sí para que la app funcione. No se enfoque en el código, sino en la historia completa del recorrido de los datos desde que la app se abre hasta que se muestra la información en pantalla.

## 7. DESAFÍO EXTRA

**Agrega un nuevo estado a la máquina de estados:** Empty Este estado debe mostrarse cuando el email del usuario sea "guest@system.local".

**Pistas:**

- Modificar UiState.
- Modificar UserViewModel.
- Modificar UserScreen.

## 8. ENTREGABLES

1. **Informe:** con capturas de pantallas pertinentes, con mínimo las siguientes capturas:
    - **Captura de pantalla:** Estructura de archivos en el panel Project donde se visualice el paquete de la aplicación.
    - **Captura de pantalla:** Simulador con la aplicación funcional y usuario cargado
    - **Enlace al Repositorio:** URL de GitHub con el commit titulado "Lab 7: Mini Aplicacion".
    - **Demás puntos** que debe contener el informe como conclusiones, recomendaciones, etc.
    - **No olvide** responder cada una de las preguntas de reflexión.

## 9. CRITERIOS DE EVALUACIÓN

- No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe.